TRABAJO FINAL - ANÁLISIS DE COMPLEJIDAD Y RECURSIVIDAD (ENTREGA FINAL)

PLANIFICADOR DE HORARIOS

ESTUDIANTES

JERÓNIMO PÉREZ MACÍAS

NICOLAS CUELLO SUAREZ

MARIANA HENAO ECHEVERRI

CURSO

ANÁLISIS Y DISEÑO DE ALGORITMOS

PROFESOR

JUAN ESTEBAN GÓMEZ TIRADO

FECHA DE ENTREGA

27/10/2026



1. **Definición del Problema**

1.1 Definición del problema

Cada semestre, una universidad tiene que decidir dónde y cuándo se dicta cada hora de clase. Cada materia tiene un profesor, un grupo de estudiantes, una cantidad de alumnos y unas horas semanales. Los salones tienen capacidad limitada y franjas en las que no están disponibles.

El horario debe ser posible: nadie puede estar en dos sitios a la vez ni pueden coincidir dos clases en el mismo salón. Además debe ser bueno: no gastar un salón grande en un grupo pequeño y respetar los horarios que prefieren los profesores.

1.2 Entradas, salidas y restricciones

Entradas

- **Materias:** id, profesor, grupo, alumnos y horas semanales.
- **Salones:** id, capacidad y disponibilidad por franja.
- **Calendario:** días y franjas de una hora por día.
- **Preferencias:** cada profesor valora cada franja con 0 (evitar), 1 (indiferente) o 2 (preferida).

Salidas

- Un horario: para cada hora de clase, el salón y la franja asignados.
- El valor de calidad del horario, F, entre 0 y 1.
- Un aviso si no existe ningún horario válido.

Restricciones

1. La capacidad del salón debe ser mayor o igual al número de alumnos.
2. El salón debe estar disponible en esa franja.
3. Un salón no puede tener dos clases en la misma franja.
4. Un profesor no puede tener dos clases en la misma franja.
5. Un grupo no puede tener dos clases en la misma franja.
6. Todas las horas de clase deben quedar programadas.
7. Cada profesor sólo puede dar una materia.

2. Solución por fuerza bruta

- Implementación funcional que resuelva el problema sin optimizaciones
- Código documentado y ejecutable

En el repositorio de GitHub: https://github.com/jeropez/Planificador-de-Horarios.git

Código de la fuerza bruta tal como está en el repositorio (src/fuerza_bruta.py, src/validaciones.py y src/calidad.py). Cada línea tiene al lado, después de #, su costo de tiempo. Cuando la línea está dentro de un ciclo, el costo ya incluye las repeticiones (por ejemplo, O(1) × n = O(n)). Al final de cada función hay una línea “Suma” que junta todos los costos. Notación: n = sesiones, B = bloques, S = salones, i = número de la vuelta, m = asignaciones recibidas (m ≤ n):

complejidad.ipynb · parte 1 de 9

```python
# src/fuerza_bruta.py
from itertools import product

from .calidad import evaluar_calidad
from .models.asignacion import Asignacion
from .validaciones import (
    cumple_r1_capacidad,
    hay_conflicto_grupo,
    hay_conflicto_profesor,
    hay_conflicto_salon,
)


def _es_valida(asignaciones: list[Asignacion], indices: dict) -> bool:
    """Verifica R1, R3, R4, R5 sobre una combinación completa."""
    contexto = indices["contexto_sesion"]  # O(1)
    vistas: list[Asignacion] = []  # O(1)

    for a in asignaciones:  # n vueltas
        ctx = contexto[a.id_sesion]  # O(1) × n = O(n)
        grupo = ctx["grupo"]  # O(1) × n = O(n)
        profesor = ctx["profesor"]  # O(1) × n = O(n)

        if hay_conflicto_salon(vistas, a.id_salon, a.id_bloque):
            # O(i) en la vuelta i (recorre las i-1 ya vistas) → 1+2+…+n = O(n²)
            return False  # O(1), se ejecuta una sola vez (sale de la función)
        if hay_conflicto_profesor(vistas, contexto, profesor.id, a.id_bloque):
            # O(i) en la vuelta i → 1+2+…+n = O(n²)
            return False  # O(1), se ejecuta una sola vez (sale de la función)
        if hay_conflicto_grupo(vistas, contexto, grupo.id, a.id_bloque):
            # O(i) en la vuelta i → 1+2+…+n = O(n²)
            return False  # O(1), se ejecuta una sola vez (sale de la función)

        vistas.append(a)  # O(1) × n = O(n)

    return True  # O(1)
# Suma de _es_valida: O(1) + O(1) + 3·O(n) + 3·O(n²) + O(n) + O(1) = O(n²)
```

complejidad.ipynb · parte 2 de 9

```python
def fuerza_bruta(
    sesiones: list,
    datos: dict,
    indices: dict,
) -> dict:
    """
    Recorre todas las combinaciones y devuelve la mejor solución válida.

    Parámetros
    ----------
    sesiones : list[Sesion]
    datos    : dict con 'bloques' y 'salones'
    indices  : dict de indices.construir_indices

    Retorna
    -------
    dict con:
        solucion              : list[Asignacion] | None
        calidad               : float
        soluciones_encontradas: int
        combinaciones_probadas: int
    """
    contexto = indices["contexto_sesion"]  # O(1)
    bloques = datos["bloques"]  # O(1)
    salones = datos["salones"]  # O(1)
    bloques_por_id = {b.id: b for b in bloques}  # O(B)

    # Todas las opciones por sesión: (bloque, salón)
    opciones_por_sesion = [  # O(n·B·S): n listas con B·S pares (bloque, salón) cada una
        [(bloque, salon) for bloque in bloques for salon in salones]
        for _ in sesiones
    ]
```

complejidad.ipynb · parte 3 de 9

```python
    mejor_solucion: list[Asignacion] | None = None  # O(1)
    mejor_calidad = -1.0  # O(1)
    soluciones_encontradas = 0  # O(1)
    combinaciones_probadas = 0  # O(1)

    for combinacion in product(*opciones_por_sesion):
        # N = (B·S)^n vueltas; cada vuelta genera una tupla de n elementos: O(n)
        combinaciones_probadas += 1  # O(1) por vuelta

        # Construir asignaciones, validando R1 en el camino
        asignaciones: list[Asignacion] = []  # O(1) por vuelta
        valida = True  # O(1) por vuelta
        for sesion, (bloque, salon) in zip(sesiones, combinacion):
            # n vueltas por combinación
            grupo = contexto[sesion.id]["grupo"]  # O(1) × n = O(n) por combinación
            if not cumple_r1_capacidad(salon, grupo):  # O(1) × n = O(n) por combinación
                valida = False  # O(1)
                break  # O(1), corta solo este for interno
            asignaciones.append(
                # O(1) × n = O(n) por combinación (crear Asignacion y añadirla)
                Asignacion(
                    id_sesion=sesion.id,
                    id_salon=salon.id,
                    id_bloque=bloque.id,
                )
            )

        if not valida:  # O(1) por vuelta
            continue  # O(1)

        if not _es_valida(asignaciones, indices):  # O(n²) por combinación (ver _es_valida)
            continue  # O(1)
```

complejidad.ipynb · parte 4 de 9

```
        soluciones_encontradas += 1  # O(1) por vuelta

        calidad = evaluar_calidad(asignaciones, indices, bloques_por_id)
            # O(n²) por combinación válida (ver evaluar_calidad)
        if calidad > mejor_calidad:  # O(1) por vuelta
            mejor_calidad = calidad  # O(1)
            mejor_solucion = list(asignaciones)  # O(n): copia la lista

    return {  # O(1)
        "solucion": mejor_solucion,
        "calidad": mejor_calidad if mejor_solucion is not None else 0.0,
        "soluciones_encontradas": soluciones_encontradas,
        "combinaciones_probadas": combinaciones_probadas,
    }
# Costo de una combinación: O(n) tupla + O(n) construir + O(n²) _es_valida + O(n²)
  evaluar_calidad + O(n) copia = O(n²)
# Suma de fuerza_bruta: O(n·B·S) preparación + (B·S)^n combinaciones × O(n²) =
  O(n²·(B·S)^n) en tiempo; espacio O(n·B·S)

# src/validaciones.py
def cumple_r1_capacidad(salon: Salon, grupo: Grupo) -> bool:
    """R1: el salón debe tener capacidad suficiente para el grupo."""
    return salon.capacidad >= grupo.cantidad_de_estudiantes  # O(1)
# Suma de cumple_r1_capacidad: O(1)


def hay_conflicto_salon(
    asignaciones: list[Asignacion],
    id_salon: str,
    id_bloque,
) -> bool:
    """R3: un salón no puede tener dos clases en el mismo bloque."""
    return any(  # m vueltas como máximo (m = asignaciones recibidas, m ≤ n)
        a.id_salon == id_salon and a.id_bloque == id_bloque  # O(1) × m = O(m)
        for a in asignaciones
    )
# Suma de hay_conflicto_salon: O(m) ≤ O(n)
```

complejidad.ipynb · parte 5 de 9

```
def hay_conflicto_profesor(
    asignaciones: list[Asignacion],
    contexto_sesion: dict,
    id_profesor,
    id_bloque,
) -> bool:
    """R4: un profesor no puede impartir dos clases en el mismo bloque."""
    for a in asignaciones:  # m vueltas (m ≤ n)
        ctx = contexto_sesion[a.id_sesion]  # O(1) × m = O(m)
        if ctx["profesor"].id == id_profesor and a.id_bloque == id_bloque:  # O(1) × m = O(m)
            return True  # O(1), una sola vez
    return False  # O(1)
# Suma de hay_conflicto_profesor: O(m) ≤ O(n)


def hay_conflicto_grupo(
    asignaciones: list[Asignacion],
    contexto_sesion: dict,
    id_grupo,
    id_bloque,
) -> bool:
    """R5: un grupo no puede recibir dos clases en el mismo bloque."""
    for a in asignaciones:  # m vueltas (m ≤ n)
        ctx = contexto_sesion[a.id_sesion]  # O(1) × m = O(m)
        if ctx["grupo"].id == id_grupo and a.id_bloque == id_bloque:  # O(1) × m = O(m)
            return True  # O(1), una sola vez
    return False  # O(1)
# Suma de hay_conflicto_grupo: O(m) ≤ O(n)

# src/calidad.py
from itertools import combinations

from .models.asignacion import Asignacion
```

complejidad.ipynb · parte 6 de 9

```
PESOS = {  # O(1), se crea una sola vez al importar el módulo
    "preferencia":    0.30,
    "misma_franja":   0.20,
    "mismo_salon":    0.20,
    "dias_distintos": 0.20,
    "uso_salones":    0.10,
}


def _franja(bloque) -> tuple[str, str]:
    """Devuelve (hora_inicio, hora_fin) del bloque."""
    return (bloque.hora_inicio, bloque.hora_fin)  # O(1)
# Suma de _franja: O(1)


def _preferencia_a_valor(p: int) -> float:
    """Mapea 0/1/2 a 0.0/0.5/1.0."""
    return {0: 0.0, 1: 0.5, 2: 1.0}.get(p, 0.5)  # O(1)
# Suma de _preferencia_a_valor: O(1)
```

complejidad.ipynb · parte 7 de 9

```
def evaluar_calidad(
    asignaciones: list[Asignacion],
    indices: dict,
    bloques_por_id: dict,
    pesos: dict | None = None,
) -> float:
    """
    Calcula la calidad normalizada de una solución.

    Parámetros
    ----------
    asignaciones : list[Asignacion]
    indices : dict
        Salida de indices.construir_indices. Se usa 'contexto_sesion'.
    bloques_por_id : dict[UUID, Bloque]
        Mapa id_bloque -> Bloque, para consultar la franja y el día.
    pesos : dict | None
        Si None, usa PESOS.

    Retorna
    -------
    float en [0, 1].
    """
    if pesos is None:  # O(1)
        pesos = PESOS  # O(1)

    if not asignaciones:  # O(1)
        return 0.0  # O(1)

    contexto = indices["contexto_sesion"]  # O(1)

    suma_pref = 0.0  # O(1)
    for a in asignaciones:  # n vueltas
        profesor = contexto[a.id_sesion]["profesor"]  # O(1) × n = O(n)
        valor = profesor.preferencias.get(a.id_bloque, 1)  # O(1) × n = O(n)
        suma_pref += _preferencia_a_valor(valor)  # O(1) × n = O(n)
    calidad_pref = suma_pref / len(asignaciones)  # O(1)
```

complejidad.ipynb · parte 8 de 9

```
    grupos: dict[tuple, list[Asignacion]] = {}  # O(1)
    for a in asignaciones:  # n vueltas
        ctx = contexto[a.id_sesion]  # O(1) × n = O(n)
        clave = (ctx["materia"].id, ctx["grupo"].id)  # O(1) × n = O(n)
        grupos.setdefault(clave, []).append(a)  # O(1) × n = O(n)

    pares_totales = 0  # O(1)
    pares_misma_franja = 0  # O(1)
    pares_mismo_salon = 0  # O(1)
    pares_dias_distintos = 0  # O(1)

    for asigs in grupos.values():  # G vueltas (G = número de grupos, G ≤ n)
        for a1, a2 in combinations(asigs, 2):
            # k(k-1)/2 pares por grupo de k sesiones; entre todos los grupos ≤ n(n-1)/2 =
            # O(n²)
            pares_totales += 1  # O(1) × O(n²) = O(n²)
            b1 = bloques_por_id[a1.id_bloque]  # O(1) × O(n²) = O(n²)
            b2 = bloques_por_id[a2.id_bloque]  # O(1) × O(n²) = O(n²)

            if _franja(b1) == _franja(b2):  # O(1) × O(n²) = O(n²)
                pares_misma_franja += 1  # O(n²) como máximo
            if a1.id_salon == a2.id_salon:  # O(1) × O(n²) = O(n²)
                pares_mismo_salon += 1  # O(n²) como máximo
            if b1.dia != b2.dia:  # O(1) × O(n²) = O(n²)
                pares_dias_distintos += 1  # O(n²) como máximo

    if pares_totales > 0:  # O(1)
        calidad_franja = pares_misma_franja / pares_totales  # O(1)
        calidad_salon = pares_mismo_salon / pares_totales  # O(1)
        calidad_dias = pares_dias_distintos / pares_totales  # O(1)
    else:
        calidad_franja = None  # O(1)
        calidad_salon = None  # O(1)
        calidad_dias = None  # O(1)
```

complejidad.ipynb · parte 9 de 9

```
    salones_usados = len({a.id_salon for a in asignaciones})
        # O(n): construye un conjunto con n elementos como máximo
    calidad_uso = 1.0 / salones_usados if salones_usados > 0 else 0.0  # O(1)

    componentes = [  # O(1): 5 elementos fijos
        (pesos["preferencia"],    calidad_pref),
        (pesos["misma_franja"],   calidad_franja),
        (pesos["mismo_salon"],    calidad_salon),
        (pesos["dias_distintos"], calidad_dias),
        (pesos["uso_salones"],    calidad_uso),
    ]

    peso_total = 0.0  # O(1)
    suma = 0.0  # O(1)
    for peso, valor in componentes:  # 5 vueltas = O(1)
        if valor is None:  # O(1)
            continue  # O(1)
        peso_total += peso  # O(1)
        suma += peso * valor  # O(1)

    if peso_total == 0.0:  # O(1)
        return 0.0  # O(1)

    return suma / peso_total  # O(1)
# Suma de evaluar_calidad: O(n) + O(n) + O(n²) + O(n) + O(1) = O(n²)
```

3. Análisis de complejidad

3.1 Complejidad temporal y espacial de la solución por fuerza bruta (Big O)

- **Notación:**

Símbolo	Significado	Valor en los casos de prueba
n	Número de sesiones a programar (len(sesiones))	2, 3 y 4
B	Número de bloques de tiempo	30
S	Número de salones	2
b = B·S	Opciones (bloque, salón) disponibles para cada sesión	60
M, P, G	Número de materias, profesores y grupos	igual a n en estos casos

- **Cómo funciona el algoritmo (resumen)**

  El algoritmo genera todas las formas posibles de asignar un par (bloque, salón) a cada sesión, usando el producto cartesiano completo (itertools.product). Para cada combinación construye las asignaciones y verifica las restricciones R1 (capacidad del salón), R3 (salón libre), R4 (profesor libre) y R5 (grupo libre). Si la combinación es válida, calcula su calidad F con evaluar_calidad y conserva la mejor. No poda ninguna rama: todas las combinaciones se generan antes de validarse.

- **Complejidad temporal**

  El número de combinaciones es $(B \cdot S)^n$, porque cada una de las n sesiones tiene B·S opciones independientes. Procesar una combinación cuesta como máximo $O(n^2)$: $O(n)$ para construir las asignaciones, $O(n^2)$ para validar los conflictos, ya que cada asignación se compara con las anteriores, y $O(n^2)$ para calcular la calidad de las combinaciones válidas. El costo de cada parte aparece anotado en el código de la sección 2.

Caso	Complejidad	Explicación
Peor caso (no hay solución, o se recorre todo el espacio)	**$O(n^2 \cdot (B \cdot S)^n)$**	Se prueban las $(B \cdot S)^n$ combinaciones, cada una con costo $O(n^2)$.
Mejor caso	Igual al peor caso	El código no tiene salida anticipada: recorre las $(B \cdot S)^n$ combinaciones y calcula la calidad de todas las válidas. Solo cambia el costo por combinación, no su número.

Interpretación:

- Es una complejidad **exponencial en n**: la base es B·S (al menos 2) y el exponente es el tamaño de entrada n.
- Para un n fijo, es un **polinomio de grado n en B·S**.

- El factor n^2 es menor frente al término exponencial y queda completamente dominado por él.

Ejemplo con los casos de prueba (B=30, S=2, b=60):

Caso	n	Combinaciones = 60^n
caso_pequeno	2	3 600
caso_mediano	3	216 000
caso_grande	4	12 960 000

- **Complejidad espacial**

Estructura	Espacio
Lista de opciones por sesión (opciones_por_sesion) y las copias internas de product	O(n · B · S)
Combinación actual, lista asignaciones y lista vistas	O(n)
Solución almacenada	O(n)
Índices (contexto_sesion, etc.)	O(M + P + G + n)

Complejidad espacial total: O(n · B · S).
itertools.product es un iterador perezoso: **no** guarda en memoria las $(B \cdot S)^n$ combinaciones, solo la combinación actual. Por eso el problema de esta solución es el **tiempo**, no la memoria.

- **Resumen**

Recurso	Big O
Tiempo (peor caso)	**$O(n^2 \cdot (B \cdot S)^n)$**: exponencial en n
Espacio	**$O(n \cdot B \cdot S)$**: polinomial

3.2 Justificación paso a paso

Esta parte deriva los resultados de la Parte 1 siguiendo el código de src/fuerza_bruta.py, src/validaciones.py, src/generar_sesiones.py e src/indices.py. Se cuentan las operaciones elementales (comparaciones, accesos a diccionario, creación de objetos) y se usa la regla habitual: los accesos a diccionarios y atributos cuestan O(1).

Paso 0. Definir el tamaño de la entrada

La variable que más influye en el costo es n, el número de sesiones. Se genera en generar_sesiones con una sesión por cada grupo de cada materia y por cada sesión semanal. Las otras dos variables son B (bloques) y S (salones). Todo el análisis se expresa en función de n, B y S.

Paso 1. Preprocesamiento (cargar datos, generar sesiones, construir índices)

- **cargar_datos:** lee el JSON y recorre una vez las listas de bloques, salones, grupos, materias y profesores, creando un objeto por elemento. Costo: O(B + S + G + M + P).
- **generar_sesiones:** tiene tres bucles anidados (materias, grupos de la materia, sesiones semanales), pero el número total de iteraciones es exactamente el número de sesiones creadas. Costo: O(n).
- **construir_indices:** construye diccionarios recorriendo materias, profesores y sesiones una vez. Cada búsqueda en un diccionario cuesta O(1). Costo: O(M + P + n).

Conclusión del paso: el preprocesamiento es **lineal** en el tamaño de la entrada. Es despreciable frente al recorrido de combinaciones que se analiza a continuación.

Paso 2. Construcción de las opciones por sesión

```
opciones_por_sesion = [
   [(bloque, salon) for bloque in bloques for salon in salones]
   for _ in sesiones
]
```

- La lista interna genera B·S parejas.
- Esa lista interna se construye una vez por cada sesión, es decir, n veces.

Costo: O(n · B · S) en tiempo y en espacio. Nótese que no se filtra nada (ni siquiera R1): cada sesión tiene las B·S opciones posibles.

Paso 3. Tamaño del espacio de búsqueda (regla del producto)

product(*opciones_por_sesion) genera una tupla por cada forma de elegir una opción para cada sesión. Como las elecciones son independientes:

`|espacio| = (B·S) × (B·S) × ... × (B·S)  (n factores)  = (B·S)^n`

Esta es la cantidad de veces que se ejecuta el cuerpo del bucle for combinacion in product(...). Es el origen del crecimiento exponencial.

Caso	n	b = B·S	$(B·S)^n$
caso_pequeno	2	60	3 600
caso_mediano	3	60	216 000
caso_grande	4	60	12 960 000

Cada sesión adicional multiplica el número de combinaciones por 60.

Paso 4. Costo de procesar UNA combinación

Dentro del bucle, para cada combinación se hacen cuatro cosas.

4a. Recibir la combinación de product. Cada iteración entrega una tupla de longitud n. Costo: O(n).

4b. Construir las asignaciones y verificar R1.

```
for sesion, (bloque, salon) in zip(sesiones, combinacion):
   ctx = contexto_sesion[sesion.id]      # O(1)
   grupo = ctx["grupo"]                  # O(1)
   if not cumple_r1_capacidad(salon, grupo):   # O(1): compara dos enteros
      valida = False
```

```
        break
    asignaciones.append(Asignacion(...))  # O(1)
```
El bucle hace como máximo n iteraciones, cada una con trabajo O(1). Costo: O(n). Si R1 falla, el bucle se corta antes, pero en el peor caso recorre todo.

4c. Validar conflictos con _es_valida.

```
for a in asignaciones:                              # n iteraciones
    if hay_conflicto_salon(vistas, ...):     return False   # recorre vistas
    if hay_conflicto_profesor(vistas, ...):  return False   # recorre vistas
    if hay_conflicto_grupo(vistas, ...):     return False   # recorre vistas
    vistas.append(a)
```
La lista vistas crece de a una asignación por iteración. En la iteración número i (empezando en 0), vistas tiene i elementos y cada una de las tres funciones de conflicto la recorre completa en el peor caso, con trabajo O(1) por elemento (una comparación y un acceso al diccionario contexto_sesion). Sumando sobre todas las iteraciones:

Comparaciones = $3 \cdot (0 + 1 + 2 + ... + (n-1)) = 3 \cdot n(n-1)/2 \rightarrow O(n^2)$

Por ejemplo, con n = 4 son como máximo $3 \cdot (0+1+2+3) = 18$ comparaciones. Las funciones usan any(...) o un return True anticipado, así que se detienen apenas encuentran un conflicto; eso solo reduce el costo en combinaciones inválidas y no cambia la cota superior.

4d. Calcular la calidad con evaluar_calidad.

```
for a in asignaciones:                  # O(n): preferencia del profesor de cada sesión
    suma_pref += _preferencia_a_valor(valor)
for a in asignaciones:                  # O(n): agrupa las sesiones por (materia, grupo)
    grupos.setdefault(clave, []).append(a)
for asigs in grupos.values():           # suma de C(k,2) pares, a lo sumo n(n-1)/2
    for a1, a2 in combinations(asigs, 2):  # O(n²) si todas las sesiones son del mismo grupo
        ...                             # cada par se compara en O(1)
salones_usados = len({a.id_salon for a in asignaciones})   # O(n)
```
Esta función solo se ejecuta para las combinaciones válidas. La cantidad de pares de sesiones del mismo grupo es a lo sumo n(n-1)/2, por lo que su costo es $O(n^2)$. En los casos de prueba cada grupo tiene una sola sesión y no hay pares, pero la cota general es $O(n^2)$.

4e. Costo total por combinación.

$O(n) + O(n) + O(n^2) + O(n^2) = O(n^2)$

Este valor es una **cota superior**. El caso en que se alcanza el n^2 completo es aquel en que la combinación sobrevive a casi todas las verificaciones, algo habitual en los casos de prueba: entre el 90 % y el 98 % de las combinaciones son válidas (ver la sección 4), así que casi todas recorren la validación completa y la evaluación de calidad.

Paso 5. Costo temporal total

Se suman los costos de los pasos anteriores:

$T(n, B, S) = T_preprocesamiento + T_opciones + (número\ de\ combinaciones) \times (costo\ por\ combinación)$

$= O(B + S + G + M + P + n) + O(n \cdot B \cdot S) + (B \cdot S)^n \times O(n^2)$

Para $B \cdot S \geq 2$ y $n \geq 1$, el último término domina a los otros dos, así que:

$$T(n, B, S) = O(n^2 \cdot (B \cdot S)^n)$$

¿Por qué es exponencial y no polinomial? Un polinomio en n tiene la forma n^k con k constante. Aquí n está en el *exponente*: $(B \cdot S)^n = 2^{n \cdot \log_2(B \cdot S)}$, que crece más rápido que cualquier polinomio en n cuando $B \cdot S \geq 2$. En cambio, si se fija n y se hace crecer B·S, el tiempo es un polinomio de grado n en B·S.

¿Qué tan ajustada es la cota? El término exponencial $(B \cdot S)^n$ es exacto en el peor caso, porque el bucle visita todas las combinaciones. El factor n^2 es una cota superior (el costo por combinación está entre $\Omega(n)$ y $O(n^2)$). Como $(B \cdot S)^n$ domina completamente, esta imprecisión no afecta la conclusión. Se comprobará con mediciones en la Parte 3.

Paso 6. Análisis por casos

- **Peor caso:** la instancia no tiene solución (o tiene solución: el algoritmo no tiene salida anticipada y recorre todo el espacio). Se procesan las $(B \cdot S)^n$ combinaciones: **$O(n^2 \cdot (B \cdot S)^n)$**.
- **Mejor caso:** el algoritmo no tiene break ni poda, así que también visita las $(B \cdot S)^n$ combinaciones, con o sin solución. Solo cambia el costo por combinación, que va de O(1), si R1 falla pronto, a $O(n^2)$. El tiempo total sigue siendo exponencial en n.

Paso 7. Complejidad espacial

Estructura	Por qué	Espacio
opciones_por_sesion	n listas, cada una con B·S parejas (Paso 2)	O(n·B·S)
Copias internas de product	product convierte cada lista de entrada en una tupla interna (un "pool") y guarda n pools de B·S elementos	O(n·B·S)
Combinación actual	Una tupla de n elementos a la vez, no se acumulan	O(n)
asignaciones y vistas	A lo sumo n objetos Asignacion cada una	O(n)
Solución guardada	Una lista de n asignaciones	O(n)
Índices y datos cargados	Diccionarios y listas de tamaño lineal (Paso 1)	O(B + S + M + P + G + n)

Sumando: O(n·B·S + B + S + M + P + G). En el modelo de datos, cada materia tiene al menos un grupo y una sesión, y cada materia tiene un solo profesor, por lo que M, P y G son del orden de n o menores, y B + S ≤ B·S para B, S ≥ 2. Entonces:
Espacio total = $O(n \cdot B \cdot S)$
Punto clave: itertools.product es un **iterador perezoso**: produce cada combinación cuando se pide y la descarta, en lugar de almacenar las $(B \cdot S)^n$. Si el código hiciera list(product(...)), el espacio sería $O(n \cdot (B \cdot S)^n)$ y la ejecución fallaría por falta de memoria mucho antes que

por tiempo. Como no lo hace, el cuello de botella es el tiempo, no la memoria.
Mejora posible: como todas las listas de opciones_por_sesion son idénticas, se podría crear una sola y repetirla n veces ([opciones] * n). Esto bajaría el espacio de las listas de opciones a O(B·S), aunque product seguiría guardando sus pools internos.

Paso 8. Resumen de la justificación

Paso	Qué se calcula	Resultado
1	Preprocesamiento	O(B + S + M + P + G + n) (lineal)
2	Opciones por sesión	O(n·B·S)
3	Número de combinaciones	$(B·S)^n$
4	Costo por combinación	$O(n^2)$
5	Tiempo total	**$O(n^2 \cdot (B·S)^n)$**
6	Casos	Peor y mejor caso exponenciales: siempre se recorren las $(B·S)^n$ combinaciones
7	Espacio total	**O(n · B · S)**

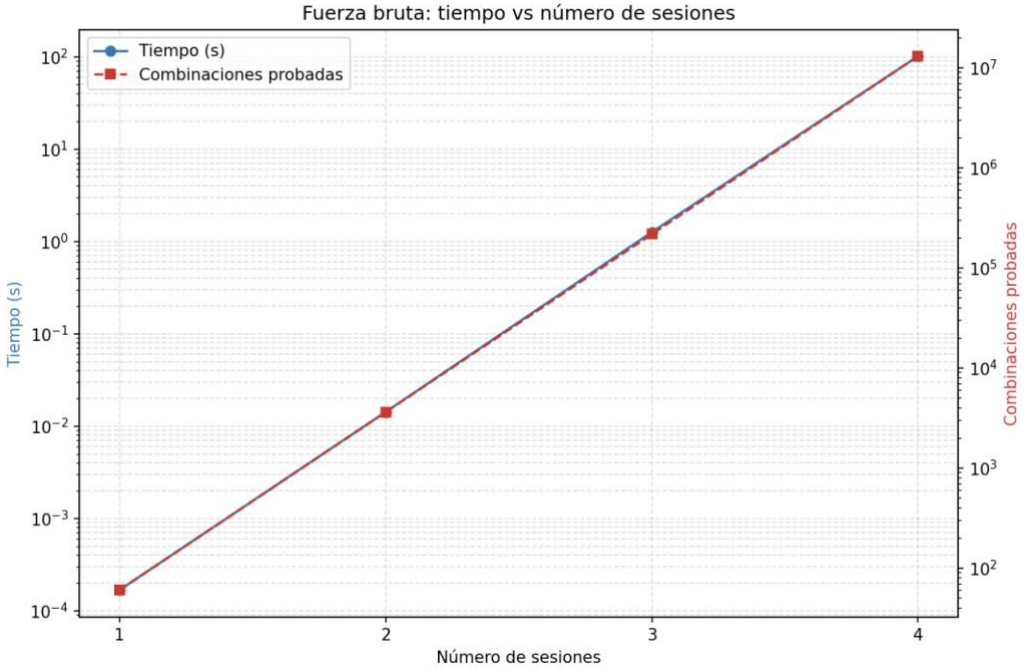
3.3 Medición empírica de tiempos con diferentes tamaños de entrada

Se midió el tiempo que tarda la fuerza bruta en cada caso de prueba con time.perf_counter(), ejecutando solo la función fuerza_bruta (sin contar la carga de datos).

caso	sesiones	combinaciones	tiempo
exp_s1	1	60	0.0002 s
caso_pequeno	2	3 600	0.0142 s
caso_mediano	3	216 000	1.2597 s
caso_grande	4	12 960 000	101.0279 s

El número de combinaciones probadas coincide exactamente con $(B·S)^n = 60^n$, donde B = 30 bloques y S = 2 salones, lo que confirma que el algoritmo recorre todo el espacio de búsqueda. Cada sesión adicional multiplica las combinaciones por 60 y el tiempo por cerca de 80 a 90 veces (de 0,0146 s a 1,29 s, y de 1,29 s a 103,4 s), lo que evidencia un crecimiento exponencial en el número de sesiones. Con solo 4 sesiones el algoritmo ya tarda casi dos minutos, por lo que la fuerza bruta no escala a horarios de tamaño real. Los tiempos absolutos dependen de la máquina: el README del repositorio reporta 0,0745 s, 5,47 s y 438,25 s para 2, 3 y 4 sesiones, con un crecimiento por sesión similar.

3.4 Gráfica comparativa (tiempo vs tamaño de entrada)



Explicación gráfica: Tiempo de ejecución y número de combinaciones probadas en función del número de sesiones (escala logarítmica). Ambas curvas son aproximadamente rectas, lo que indica crecimiento exponencial: cada sesión adicional multiplica el número de combinaciones por 60 (B·S = 60) y el tiempo por cerca de 80 a 90. Con 4 sesiones el algoritmo tarda unos 103 s, lo que muestra que la fuerza bruta no escala a horarios de tamaño real.

4. Casos de prueba y salidas

4.1 Resumen de la ejecución

La siguiente tabla resume la ejecución de la fuerza bruta en cada caso de prueba (B = 30 bloques y S = 2 salones). El número de combinaciones probadas coincide con 60^n y la calidad F = 1,0 indica que existe al menos un horario válido con la mejor valoración posible.

Caso	Sesiones (n)	Combinaciones probadas	Soluciones válidas	% válidas	Calidad F
caso_pequeno	2	3 600	3 540	98,3 %	1,0
caso_mediano	3	216 000	205 320	95,1 %	1,0
caso_grande	4	12 960 000	11 703 240	90,3 %	1,0

4.2 Horario resultante por caso

Para cada caso se muestra el mejor horario encontrado, con la materia, el profesor, el grupo, el salón, el día y la franja de cada sesión.

4.2.1 Caso pequeño (2 sesiones)

Materia	Profesor	Grupo	Salón	Día	Franja
Cálculo en Varias Variables	Profesor_01	G01	11-103	Lunes	08:00-10:00
Análisis y Diseño de Algoritmos	Profesor_02	G02	11-103	Martes	14:00-16:00

4.2.2 Caso mediano (3 sesiones)

Materia	Profesor	Grupo	Salón	Día	Franja
Cálculo en Varias Variables	Profesor_01	G01	13-109	Lunes	10:00-12:00
Análisis y Diseño de Algoritmos	Profesor_02	G02	13-109	Martes	06:00-08:00
Modelos de Datos	Profesor_03	G03	13-109	Lunes	16:00-18:00

4.2.3 Caso grande (4 sesiones)

Materia	Profesor	Grupo	Salón	Día	Franja
Cálculo en Varias Variables	Profesor_01	G01	8-100	Lunes	08:00-10:00
Análisis y Diseño de Algoritmos	Profesor_02	G02	8-100	Lunes	06:00-08:00
Modelos de Datos	Profesor_03	G03	8-100	Lunes	12:00-14:00
Ecuaciones Diferenciales	Profesor_04	G04	8-100	Lunes	16:00-18:00

El programa entrega este horario como un diccionario por salón, con una lista por día y seis franjas por día (src/salida.py); aquí solo se muestran las franjas ocupadas. Un ejemplo completo de la salida está en datos/salida_caso_pequeno.json del repositorio.

5. Solución greedy

5.1 Decisión local óptima

Un algoritmo greedy construye la solución paso a paso: en cada paso elige la mejor opción disponible en ese momento y nunca la cambia después. Para este problema

se toman dos decisiones locales, una sobre el orden de las sesiones y otra sobre su ubicación.

Decisión 1: qué sesión se asigna primero. Se asigna primero la sesión más restringida, es decir, la que tiene menos opciones (bloque, salón) que cumplan R1 con el tablero vacío. Si dos sesiones empatan, va primero la del grupo con más estudiantes. La razón es que un grupo grande solo cabe en los salones de mayor capacidad; si se deja para el final, esos salones pueden estar ocupados y la sesión se quedaría sin lugar.

Decisión 2: dónde se asigna. Entre las opciones que cumplen R1, R3, R4 y R5 con lo ya asignado, se elige la que da la mayor calidad F, calculada con evaluar_calidad sobre las asignaciones hechas más la candidata. Es la misma función de calidad de la fuerza bruta, así que ambas soluciones optimizan el mismo objetivo y sus resultados son comparables. Si hay un empate de calidad, gana el salón con menos capacidad sobrante (no se gasta un salón grande en un grupo pequeño) y, si aún hay empate, la primera opción en el orden original de bloques y salones.

La decisión es local porque solo considera la sesión actual y las ya asignadas, no las que faltan. Una vez colocada, una sesión no se mueve (no hay retroceso). Si una sesión no tiene ninguna opción factible, queda sin asignar, R6 no se cumple y el resultado lo informa con completa = False.

5.2 Cómo funciona el algoritmo

El código está en src/greedy.py y usa los mismos módulos que la fuerza bruta (cargador, generar_sesiones, indices y calidad). Recibe las sesiones, los datos y los índices, y devuelve la solución, su calidad, las sesiones sin asignar y el número de candidatos evaluados. Los pasos son:

Paso 1. Tableros de ocupación. Se crean tres conjuntos: (profesor, bloque) para R4, (grupo, bloque) para R5 y (salón, bloque) para R3. Saber si hay conflicto es una búsqueda en un conjunto, de costo O(1). La fuerza bruta, en cambio, recorre la lista vistas completa en cada comprobación.

Paso 2. Opciones factibles. Para una sesión se recorren todos los pares (bloque, salón) y se descartan los que rompen R1, R3, R4 o R5. R2 no se comprueba por separado porque queda cubierta por R3, igual que en la fuerza bruta.

Paso 3. Orden de las sesiones. Se calculan las opciones factibles de cada sesión con el tablero vacío y se ordenan de menor a mayor número de opciones (desempate: más estudiantes primero). El orden se calcula una sola vez.

Paso 4. Elección de la mejor opción. Para cada sesión, en ese orden, se recalculan sus opciones (el tablero ya cambió), se evalúa la calidad de cada candidata y se elige la mejor según la Decisión 2.

Paso 5. Registro. La opción elegida se agrega a la solución y se marcan el profesor, el grupo y el salón como ocupados en ese bloque. Si no había opciones, la sesión pasa a la lista de no asignadas.

5.3 Complejidad de cada parte del código

Se usa la notación de la sección 3.1: n = sesiones, B = bloques, S = salones y b = B·S. Además, i es el número de sesiones ya asignadas (i ≤ n). Como en la sección 2, cada línea lleva al lado, después de #, su costo, y las líneas dentro de un ciclo ya incluyen las repeticiones. El código está resumido. Según el Paso 4d de la sección 3.2, evaluar_calidad con m asignaciones cuesta O(m²) en el peor caso.

5.3.1 Preparación

```
bloques = datos["bloques"]                    # O(1)
salones = datos["salones"]                    # O(1)
contexto = indices["contexto_sesion"]         # O(1)
bloques_por_id = {b.id: b for b in bloques}   # O(B)
prof_ocupado = set()                          # O(1)
grupo_ocupado = set()                         # O(1)
salon_ocupado = set()                         # O(1)
```

Suma: O(B)

5.3.2 Opciones factibles de una sesión

```
ctx = contexto[sesion.id]                       # O(1)
id_prof = ctx["profesor"].id                    # O(1)
id_grupo = ctx["grupo"].id                      # O(1)
n_estudiantes = ctx["grupo"].cantidad_de_estudiantes  # O(1)
ops = []                                        # O(1)
for bloque in bloques:                          # B vueltas
    if (id_prof, bloque.id) in prof_ocupado:    # O(1) × B = O(B)
        continue
    if (id_grupo, bloque.id) in grupo_ocupado:  # O(1) × B = O(B)
        continue
    for salon in salones:                       # S vueltas por bloque
        if salon.capacidad < n_estudiantes:     # O(1) × B·S = O(b)
            continue
        if (salon.id, bloque.id) in salon_ocupado:  # O(b)
            continue
        ops.append((bloque, salon))             # O(b)
return ops                                      # O(1)
```

Suma: O(B) + O(b) = O(b)

Costo: O(b) = O(B·S) por llamada, en tiempo y en espacio (la lista ops). Esta función se llama n veces para calcular el orden y n veces más en el bucle principal.

5.3.3 Orden de las sesiones

```
orden = sorted(
    sesiones,
    key=lambda s: (
        len(opciones_factibles(s)),   # O(b) × n llamadas = O(n·b)
        -contexto[s.id]["grupo"].cantidad_de_estudiantes,  # O(n)
    ),
)                                     # ordenar n claves: O(n log n)
```

Suma: O(n·b + n log n)

5.3.4 Bucle principal (elección de la mejor opción)

```
for sesion in orden:                                  # n vueltas
    ops = opciones_factibles(sesion)                  # O(b)
    if not ops:                                       # O(1)
        no_asignadas.append(sesion)                   # O(1)
        continue
    for pos, (bloque, salon) in enumerate(ops):       # hasta b vueltas
        candidata = Asignacion(...)                   # O(1)
        calidad_parcial = evaluar_calidad(
            solucion + [candidata], indices, bloques_por_id
        )                                             # O(i) + O(i²) = O(i²)
        desperdicio = salon.capacidad - n_estudiantes  # O(1)
        clave = (-calidad_parcial, desperdicio, pos)   # O(1)
        if mejor_clave is None or clave < mejor_clave: # O(1)
            mejor_clave = clave                       # O(1)
            mejor_op = (bloque, salon)                # O(1)
    solucion.append(Asignacion(...))                  # O(1)
    prof_ocupado.add((id_prof, bloque.id))            # O(1)
    grupo_ocupado.add((id_grupo, bloque.id))          # O(1)
    salon_ocupado.add((salon.id, bloque.id))          # O(1)
```

Suma: $n \times (O(b) + b \times O(n^2)) = O(n^3 \cdot b)$

Costo por candidato: la línea que domina es evaluar_calidad. Copiar la lista solucion + [candidata] cuesta O(i) y evaluar_calidad sobre i + 1 asignaciones cuesta $O(i^2)$ en el peor caso (pares de sesiones del mismo grupo y materia), por lo que cada candidato cuesta $O(n^2)$ como cota. Cada sesión tiene como máximo b candidatos, así que cuesta $O(b \cdot n^2)$, y las n sesiones suman $O(n^3 \cdot b)$.

En los casos de prueba cada grupo tiene una sola sesión y no hay pares, de modo que evaluar_calidad cuesta O(i) y el bucle completo cuesta $O(n^2 \cdot b)$.

5.3.5 Resultado final

Al terminar se calcula la calidad de la solución completa con una última llamada a evaluar_calidad. Costo: $O(n^2)$.

5.4 Complejidad de la solución en general

Complejidad temporal

Sumando las cinco partes:

$$T(n, B, S) = O(B) + O(n \cdot b + n \log n) + O(n^3 \cdot b) + O(n^2)$$

A esto se suma el preprocesamiento (cargar datos, generar sesiones y construir índices), que es el mismo de la fuerza bruta y lineal en el tamaño de la entrada (Paso 1 de la sección 3.2). El término $O(n^3 \cdot b)$ domina a los demás, así que:

$$T(n, B, S) = O(n^3 \cdot B \cdot S)$$

Interpretación:

- Es una complejidad **polinomial**: cúbica en n y lineal en B·S. Una sesión adicional ya no multiplica el tiempo por 60, solo lo aumenta de forma gradual.
- El costo lo domina evaluar_calidad, que se llama una vez por candidato, como máximo $n \cdot b$ veces. Cuando ningún grupo tiene dos sesiones de la misma materia (como en los casos de prueba), cada llamada cuesta O(n) y el total baja a $O(n^2 \cdot b)$.
- No hay salida anticipada: se evalúan todas las opciones factibles de cada sesión. Por eso el mejor y el peor caso tienen el mismo orden de crecimiento, salvo que una sesión se quede sin opciones y se omita su evaluación.

Complejidad espacial

Los tres conjuntos de ocupación guardan como máximo n elementos cada uno: O(n). La lista de opciones de una sesión ocupa O(b) y solo existe una a la vez. Las listas orden y solucion ocupan O(n), la copia solucion + [candidata] es temporal y de tamaño O(n), y bloques_por_id ocupa O(B). **Espacio total: O(n + B·S).** La fuerza bruta usa O(n · B · S) porque product guarda n copias de la lista de opciones (Paso 7).

Comparación con la fuerza bruta

La siguiente tabla resume los resultados de las secciones 3 y 5.

Recurso	Fuerza bruta	Greedy
Tiempo (peor caso)	$O(n^2 \cdot (B \cdot S)^n)$, exponencial	$O(n^3 \cdot B \cdot S)$, polinomial
Espacio	O(n · B · S)	O(n + B·S)
¿Garantiza el óptimo?	Sí	No

La siguiente tabla compara el número de combinaciones que prueba la fuerza bruta con el número de candidatos que evalúa greedy en cada caso de prueba (B = 30, S = 2, b = 60). Los candidatos de greedy salen de la variable candidatos_evaluados de src/greedy.py y quedan muy cerca de la cota n · b.

Caso	n	Combinaciones (fuerza bruta)	Candidatos evaluados (greedy)	Cota n · b
caso_pequeno	2	3 600	119	120
caso_mediano	3	216 000	177	180
caso_grande	4	12 960 000	234	240

Con n = 4 la fuerza bruta prueba más de 12 millones de combinaciones y greedy evalúa 234 candidatos. A cambio, como las decisiones no se revisan, greedy no garantiza el óptimo: una elección buena en el momento puede ocupar un bloque que otra sesión necesitaba y el algoritmo no puede corregirlo. En ese caso la calidad F puede quedar por debajo del máximo.

5.5 ¿Por qué greedy no garantiza el óptimo?

El script analisis/contraejemplo_greedy.py compara greedy con un óptimo exhaustivo en 3 000 instancias pequeñas. La peor encontrada tiene 2 salones (capacidad 40 y 20), 3 grupos (35, 35 y 25 estudiantes) y 4 bloques: greedy obtiene **F = 0,4875** y el óptimo **F = 0,9625**. Las primeras sesiones toman el salón grande en los bloques que más le convenían a ellas, y las que vienen después ya no pueden quedar juntas ni en el mismo salón. Como greedy no retrocede, no puede corregirlo.

6. Solución con backtracking

6.1 Estrategia y podas

El código está en src/backtracking.py. Recorre en profundidad el árbol de decisiones: en cada nivel asigna una sesión a un par (bloque, salón), baja al siguiente nivel y, al volver, deshace la asignación para probar la siguiente opción. La calidad F se calcula en las hojas (horarios completos) y se guarda la mejor, igual que en la fuerza bruta. Las podas son:

Poda	Qué hace
P1. Restricciones duras	Solo prueba opciones que cumplen R1, R3, R4 y R5, revisadas con conjuntos de ocupación en O(1).
P3. Forward checking	Antes de bajar, verifica que cada sesión pendiente siga teniendo al menos una opción. Si alguna se queda sin opciones, descarta la rama.
P4. MRV (menos valores restantes)	En cada nivel elige primero la sesión con menos opciones. Reduce el árbol sin perder el óptimo.

Con buscar_optimo=True recorre todo el árbol podado y devuelve el óptimo de F; con buscar_optimo=False se detiene en la primera solución válida.

6.2 Complejidad

- **Nodos:** en el peor caso cada sesión puede ir a cualquiera de las b = B·S opciones, así que el árbol tiene $O(b^n)$ nodos.
- **Costo por nodo:** MRV y forward checking recalculan las opciones de las sesiones pendientes: $O(n \cdot b)$. En las hojas, evaluar_calidad cuesta $O(n^2)$.
- **Temporal (peor caso):** $O(b^n \cdot n \cdot b) = O(n \cdot (B \cdot S)^{n+1})$. Las podas no cambian la cota, pero sí el trabajo real.
- **Espacial:** la recursión tiene profundidad n y cada nivel guarda su lista de opciones O(b) y las pendientes O(n): $O(n \cdot (b + n))$.

6.3 Resultados

n	Nodos (backtracking)	Combinaciones (fuerza bruta)	Tiempo backtracking	Tiempo fuerza bruta
2	3 541	3 600	14,8 ms	20,9 ms
3	205 381	216 000	1,0 s	1,4 s
4	11 515 381	12 960 000	66,7 s	117,4 s

Con 30 bloques y pocas sesiones casi todas las combinaciones son válidas, así que la poda descarta poco (explora cerca del 89 % del espacio). Aun así es unas 1,8 veces más rápido que la fuerza bruta porque revisa cada restricción en O(1) al asignar. La poda rinde mucho más cuando el problema está ajustado (pocos salones para muchas sesiones).

7. Solución con divide y vencerás

7.1 Cómo dividir el problema en subproblemas independientes

La clave es la restricción R7 y el modelo de datos: **cada profesor da una sola materia y cada grupo ve una sola materia**. Por eso dos sesiones de materias distintas:

- nunca comparten profesor (R4 no las relaciona);
- nunca comparten grupo (R5 no las relaciona);
- nunca forman un par en la calidad F (los pares de misma franja, mismo salón y días distintos son siempre del mismo grupo).

Lo único que relaciona materias distintas es R3 (no pueden usar el mismo salón en el mismo bloque) y el término global de uso de salones. Así, **cada materia es un subproblema independiente** y solo hay que coordinar los salones al combinar. Dividir el calendario por mitades de tiempo no sirve: habría que decidir antes qué sesiones van a cada mitad (2^n formas) y el criterio de días distintos relaciona las dos mitades.

7.2 Fases

Dividir. Se separan las sesiones por materia: M subproblemas de s sesiones cada uno ($n = M \cdot s$).

Vencer. Cada materia se resuelve de forma exacta. Todas sus sesiones comparten profesor, así que van en bloques distintos y nunca chocan entre sí por salón; lo mejor para el criterio de mismo salón es dejar cada grupo en un único salón. Solo hay que escoger qué bloques usa cada grupo, y como las sesiones de un grupo son intercambiables son combinaciones C(B, s), no permutaciones. El puntaje es exactamente la parte de F que depende de esa materia. Con ramificación y poda se guardan las K = 50 mejores alternativas.

Combinar. Se unen recursivamente las mitades del conjunto de materias. Si en un bloque hay más clases que salones con capacidad suficiente, se **repara** cambiando una materia a su siguiente alternativa (la que menos puntaje pierde). Como el uso de salones es global, también se **ajusta**: si cambiar una materia de alternativa reduce las clases simultáneas y F sube, se hace el cambio. Al final, cada grupo intenta quedarse en un solo salón.

Certificado de óptimo. Como las materias no dependen entre sí, la suma de sus mejores puntajes más el máximo del uso de salones es una cota superior de F. Si la solución alcanza esa cota, está demostrado que es el óptimo global (optimo_garantizado = True).

7.3 Implementación (src/divide_venceras.py, resumida)

```
# VENCER: cada materia por separado
for m in materias:                                  # M subproblemas
    alternativas[m] = mejores_alternativas(m)       # O(C(B,s) · s²) cada uno

# DIVIDIR + COMBINAR: recursión sobre el conjunto de materias
def resolver(ms):                                   # T(M)
    if len(ms) <= 1:                                # caso base: O(1)
        return ms
    mitad = len(ms) // 2
    izq = resolver(ms[:mitad])                      # T(M/2)
    der = resolver(ms[mitad:])                      # T(M/2)
    return combinar(izq, der)                       # O(n): choques por bloque
```

7.4 Análisis de complejidad (Teorema Maestro)

Vencer: cada materia evalúa C(B, s) combinaciones de bloques y cada una suma s² pares: O(M · C(B, s) · s²). Con s fijo es lineal en M; por ejemplo, con s = 2 cada materia evalúa C(30, 2) = 435 combinaciones.

Combinar: resolver parte el conjunto en dos mitades y combinar revisa la ocupación de todos los bloques en O(n) = O(s · M):

$$T(M) = 2 \cdot T(M/2) + \Theta(M)$$

Teorema Maestro con a = 2, b = 2 y f(M) = Θ(M): como M^($\log_2$ 2) = M = Θ(f(M)), es el **caso 2**, y T(M) = Θ(M log M) = Θ(n log M).

Temporal total: O(M · C(B, s) · s² + n · log M) cuando no hay choques. Cada reparación o ajuste prueba hasta K alternativas por materia con costo O(n), así que suma O(R · M · K · n), con R el número de cambios (R = 0 en los casos pequeños y medianos; ver 7.5).

Espacial: O(M · K · s + n) para las alternativas guardadas y la solución.

El exponente pasa de n (todas las sesiones, en la fuerza bruta) a s (las sesiones de una materia).

7.5 Resultados

Caso	n	Calidad F	¿Óptimo certificado?	Reparaciones / ajustes	Tiempo
caso_pequeno	2	1,000	Sí	0 / 0	0,2 ms
caso_mediano	3	1,000	Sí	0 / 0	0,2 ms
caso_grande	4	1,000	Sí	0 / 0	0,2 ms
caso_multi_pequeno	4	1,000	Sí	0 / 0	0,7 ms
caso_multi_mediano	8	1,000	Sí	0 / 1	1,8 ms
caso_multi_grande	12	0,963	Sí	0 / 2	16 ms

Los casos multi tienen varias sesiones por materia, así que los criterios por pares sí cuentan. En caso_multi_grande (n = 12) la solución alcanza la cota superior: es el óptimo global en un tamaño donde la fuerza bruta necesitaría unas 10^{23} combinaciones. Además, analisis/validar_divide_venceras.py lo comparó con el

óptimo exhaustivo en 717 instancias pequeñas: igualó el óptimo en el 95,7 %, no produjo ninguna solución inválida y nunca certificó como óptima una solución que no lo era.

8. Solución con programación dinámica

8.1 Por qué aplica

R3, R4 y R5 solo relacionan sesiones del **mismo bloque**; dos sesiones en bloques distintos nunca chocan. Por eso el problema se puede resolver bloque a bloque recordando solo qué sesiones ya se programaron. Tiene **subestructura óptima** (la mejor forma de terminar depende solo de las sesiones que faltan) y **subproblemas solapados** (distintos órdenes de colocar las mismas sesiones llegan al mismo estado).

8.2 Formulación

- **Estado:** dp[k][mask] = máxima preferencia acumulada usando los bloques 0..k−1, donde mask es el conjunto de sesiones ya ubicadas (2^n valores).
- **Transición:** en el bloque k se elige un subconjunto T de las sesiones pendientes que quepa en él (profesores distintos, grupos distintos y un salón con capacidad para cada una): dp[k+1][mask ∪ T] = máx(dp[k][mask] + valor(k, T)).
- **Respuesta:** dp[B][todas], que garantiza R6. El horario se reconstruye hacia atrás con punteros.
- **Modelo restringido:** la DP optimiza la preferencia de los profesores, que se suma por (sesión, bloque). Los criterios por pares y el uso de salones no son aditivos; incluirlos obligaría a guardar en el estado la franja, el salón y el día de cada sesión, y los estados volverían a explotar.

8.3 Implementación (src/dinamica.py, resumida)

```
for k in range(B):                          # B bloques
    for mask, v in dp.items():              # hasta 2^n estados
        comp = todas ^ mask                 # sesiones pendientes
        sub = comp
        while sub:                          # subconjuntos de comp
            if factible[sub]:               # O(1), precalculado
                nuevo[mask | sub] = max(nuevo[mask | sub], v + valor(k, sub))
            sub = (sub - 1) & comp
    dp = nuevo
```

8.4 Análisis de complejidad

- **Temporal:** en cada bloque, cada máscara recorre los subconjuntos de su complemento. Sumando sobre todas las máscaras, Σ 2^(n−|mask|) = 3^n, así que el total es **$O(B \cdot 3^n)$**, más $O(2^n \cdot n)$ para precalcular qué subconjuntos caben en un bloque.
- **Espacial:** $O(2^n)$ para la tabla de subconjuntos factibles y $O(B \cdot 2^n)$ para los punteros de reconstrucción: **$O(B \cdot 2^n)$**.
- El Teorema Maestro no aplica: es un algoritmo iterativo sobre una tabla, no una recurrencia que divide el problema en partes de tamaño n/b.

8.5 Resultados

n	6	8	10	12	14
Tiempo DP	7,4 ms	55,0 ms	381,6 ms	2,5 s	16,5 s

Cada 2 sesiones más el tiempo se multiplica por unas 7 a 8 veces, cerca del factor $3^2 = 9$ esperado: el límite práctico es $n \approx 14$. En los casos multi la DP obtiene siempre la máxima preferencia (8,0 frente a 7,0 de greedy en caso_multi_mediano), pero su calidad F es menor (0,700) porque no optimiza los criterios por pares.

9. Comparativa final

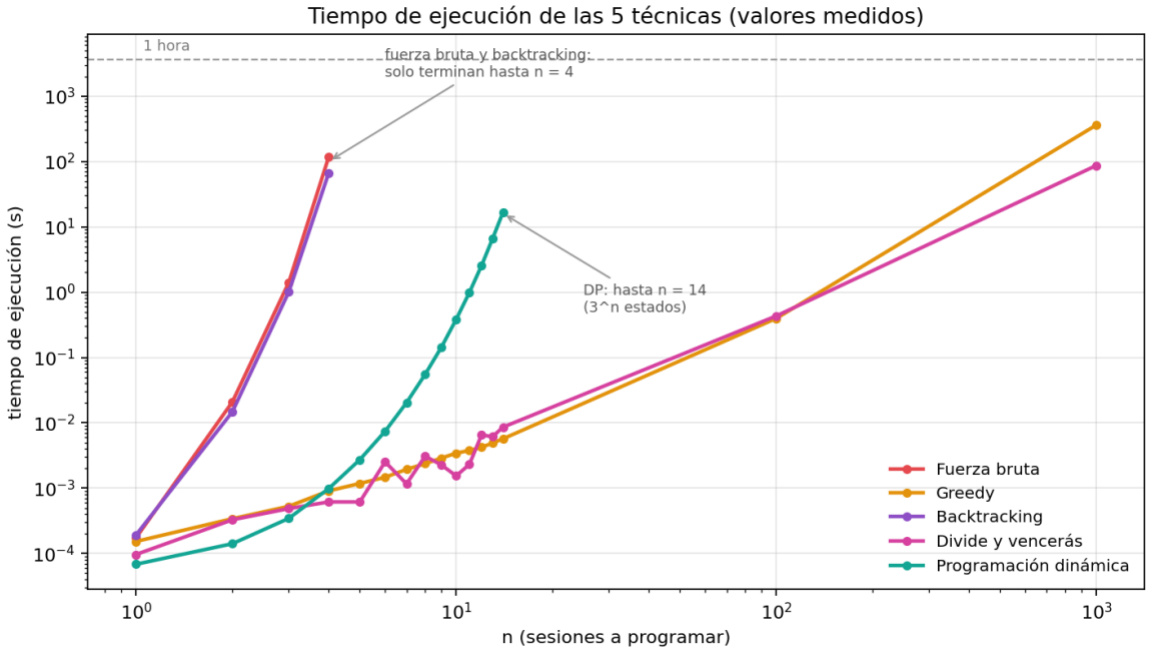
9.1 Tabla comparativa de las 5 técnicas

n es el número de sesiones a programar. Las instancias de n = 100 y n = 1000 tienen materias de 2 sesiones, 30 bloques y salones suficientes (5 y 50). Los tiempos de greedy y divide y vencerás son medidos (src/comparativa.py). Fuerza bruta, backtracking y programación dinámica no terminan con esos tamaños: su tiempo se estima con el tiempo medido en el mayor n que sí terminaron, multiplicado por lo que crece su fórmula.

Técnica	Complejidad temporal	Complejidad espacial	¿Solución óptima?	Tiempo real (n=100)	Tiempo real (n=1000)
Fuerza bruta	$O(n^2 \cdot (B \cdot S)^n)$	$O(n \cdot B \cdot S)$	Sí	$\approx 10^{208}$ años (estimado; inviable)	$\approx 10^{3168}$ años (estimado; inviable)
Greedy	$O(n^3 \cdot B \cdot S)$	$O(n + B \cdot S)$	No	396 ms (medido, F = 0,828)	6,0 min (medido, F = 0,819)
Backtracking	$O(n \cdot (B \cdot S)^{n+1})$ peor caso	$O(n \cdot (B \cdot S + n))$	Sí	$\approx 10^{207}$ años (estimado; inviable)	$\approx 10^{3167}$ años (estimado; inviable)
Divide y vencerás	$O(M \cdot C(B,s) \cdot s^2 + n \log M)$ + reparaciones	$O(M \cdot K \cdot s + n)$	Sí cuando alcanza la cota (lo certifica)	432 ms (medido, F = 0,916)	1,4 min (medido, F = 0,902)
Programación dinámica	$O(B \cdot 3^n)$	$O(B \cdot 2^n)$	Sí, pero solo en preferencia	$\approx 10^{35}$ años (estimado; inviable)	$\approx 10^{464}$ años (estimado; inviable)

Notación: n = sesiones, B = bloques (30), S = salones, M = materias, s = sesiones por materia, K = alternativas guardadas por materia (50). Además del tiempo, la DP necesitaría 2^{100} estados de memoria con n = 100.

9.2 Gráfica de tiempos de ejecución



Tiempo medido de las 5 técnicas en función de n (escala logarítmica en ambos ejes). Fuerza bruta y backtracking suben casi en vertical y solo terminan hasta n = 4; la DP crece como 3^n y llega a n = 14; greedy y divide y vencerás crecen de forma polinomial y llegan a n = 1000. Datos: datos/benchmark_tecnicas.json y datos/comparativa_resultados.json; gráfica: docs/comparativa_tiempos.png.

9.3 Calidad del horario

Caso (n)	Fuerza bruta	Greedy	Backtracking	Divide y vencerás	Prog. dinámica
caso_multi_pequeno (4)	1,000	1,000	1,000	1,000	0,600
caso_multi_mediano (8)	—	0,963	—	1,000	0,700
caso_multi_grande (12)	—	0,938	—	0,963	0,733
n = 100	—	0,828	—	0,916	—
n = 1000	—	0,819	—	0,902	—

— = la técnica no termina en un tiempo razonable con ese tamaño.

9.4 ¿Qué técnica es la más adecuada?

Para este problema la más adecuada es **divide y vencerás**. Aprovecha la estructura del problema (materias independientes), así que su costo crece casi linealmente con el número de materias en lugar de exponencialmente con n. En todos los casos obtuvo la mejor calidad F entre las técnicas que escalan, terminó en menos de medio segundo con n = 100 y en minuto y medio con n = 1000 (greedy tardó 6 minutos), y es la única que puede **demostrar** que su horario es óptimo sin recorrer todo el espacio.

Greedy es la alternativa cuando una materia tiene muchas sesiones (el subproblema de divide y vencerás vuelve a ser grande) o cuando hay muy pocos salones: es polinomial y siempre rápido, aunque sin garantía de óptimo. **Fuerza bruta y backtracking** solo sirven para instancias diminutas, como referencia para validar a las demás. **La DP** es exacta, pero solo para la preferencia y hasta $n \approx 14$.

10. Conclusiones

10.1 Qué aprendimos al aplicar cada técnica al mismo problema

- **Fuerza bruta:** es la referencia correcta y fácil de verificar, pero el crecimiento $(B \cdot S)^n$ hace que con solo 4 sesiones ya tarde minutos.
- **Greedy:** una buena regla local (la sesión más restringida primero) da horarios muy buenos en milisegundos, pero encontramos instancias donde queda lejos del óptimo.
- **Backtracking:** la poda solo ayuda si descarta ramas; con datos holgados casi todo es válido y el árbol sigue siendo exponencial.
- **Divide y vencerás:** lo importante no es el algoritmo sino encontrar la división correcta. Dividir por materia funciona porque R7 hace a las materias independientes; dividir por tiempo no.
- **Programación dinámica:** cambia la base del exponente (de 60^n a 3^n), pero obliga a simplificar el objetivo para que sea aditivo. Elegir bien el estado es la parte difícil.

10.2 Escenarios reales para cada paradigma

- **Fuerza bruta:** validar otras soluciones en casos pequeños, o problemas con muy pocas opciones (por ejemplo, probar todas las contraseñas de 4 dígitos).
- **Greedy:** cuando se necesita una respuesta inmediata y buena, como asignar tareas a servidores, dar cambio con monedas o rutas en tiempo real.
- **Backtracking:** problemas de restricciones con mucha poda, como sudokus, crucigramas o la planificación de turnos con reglas estrictas.
- **Divide y vencerás:** cuando el problema se separa en partes independientes: ordenamiento (merge sort), procesamiento en paralelo o planificar cada facultad por separado.
- **Programación dinámica:** problemas con subproblemas repetidos, como rutas más cortas, la mochila, alinear secuencias de ADN o asignar recursos por etapas.

10.3 Limitaciones encontradas

- Fuerza bruta y backtracking (en modo óptimo) son inviables para $n > 4$.
- Greedy no garantiza el óptimo y, como no retrocede, puede dejar sesiones sin asignar si una decisión temprana las bloquea.
- El backtracking solo poda por factibilidad, no por calidad; con datos holgados descarta poco.
- La DP solo optimiza la preferencia y su memoria crece como 2^n, así que llega hasta $n \approx 14$.
- Divide y vencerás depende de que cada materia tenga pocas sesiones; con pocos salones necesita más reparaciones y puede alejarse del óptimo.

- Los tiempos de n = 100 y n = 1000 de fuerza bruta, backtracking y DP son estimaciones: ejecutarlos tomaría más que la edad del universo.

10.4 Código fuente

Repositorio: https://github.com/jeropez/Planificador-de-Horarios.git. Las cinco técnicas están en src/fuerza_bruta.py, src/greedy.py, src/backtracking.py, src/divide_venceras.py y src/dinamica.py, todas con la misma interfaz. src/comparativa.py ejecuta la comparativa de n = 100 y n = 1000 y genera la gráfica (python -m src.comparativa). El README explica cómo ejecutar cada parte, y interfaz/index.html muestra los horarios y las gráficas de forma interactiva.